

36. Reprezentace a uložení strukturovaných dat, serializace, relační datový model, SQL, transakce

36. Strukturovaná data, relační model, SQL, transakce

SZZ okruh 36 (FIT BUT). Od ukládání strukturovaných dat po databázové transakce.

Shrnutí

Strukturovaná data a serializace

- **Struktura** (pevný počet pojmenovaných hodnot různých typů, „záznam“) × **kolekce** (proměnný počet hodnot téhož typu, „seznam“).
- **Serializace** = převod struktur/objektů do přenositelného formátu (JSON, XML); **deserializace** = zpětná rekonstrukce.

Relační model

- **Relace** = podmnožina kartézského součinu domén = **tabulka** (schéma = záhlaví, tělo = řádky/n-tice).
- **Klíče**: kandidátní (jednoznačný + minimální), **primární** (NOT NULL), **cizí** (odkaz na PK jiné relace).
- **Relační algebra**: projekce (SELECT sloupců), selekce (WHERE), spojení (JOIN), množinové operace.

SQL

- **DDL** (CREATE/ALTER/DROP), **DML** (INSERT/UPDATE/DELETE), **DQL** (SELECT), **DCL** (GRANT/REVOKE), **TCL** (COMMIT/ROLLBACK).
- SELECT + WHERE + GROUP BY + HAVING; **JOINy**: INNER, LEFT/RIGHT OUTER, NATURAL, CROSS.
- Pohledy (view), kurzory, uložené procedury, trigger.

Transakce (ACID)

- **Atomicity** (vše/nic), **Consistency** (zachování integrity), **Isolation** (nezávislost), **Durability** (trvalost i při výpadku).
- BEGIN/COMMIT/ROLLBACK; business transakce = více DB transakcí (OLTP).

Návrh schématu viz [[okruhy/35-konceptualni-modelovani-db]]; relace v matematice viz [[okruhy/16-mnoziny-relace-zobrazeni]]; transakce na úrovni OS viz [[okruhy/39-planovani-synchronizace-procesu]].

Související syntéza

- [[synthesis/relace-napric-obory|Relace × graf × tabulka]] — syntéza

- [[synthesis/transakce-acid-db-os | Transakce a ACID: DB × OS]] — syntéza

Časté otázky u zkoušky

Na co se u tohoto okruhu typicky ptají. Plné odpovědi níže.

Serializace □ [[#Strukturovaná data a serializace]] - *Serializace vs. deserializace?* □ Převod struktury/objektu do řetězce (JSON/XML) × zpětná rekonstrukce. - *Struktura vs. kolekce?* □ Záznam pevných pojmenovaných polí × proměnný seznam stejného typu.

Relační model □ [[#Relační model]] - *Definice relace?* □ Podmnožina kartézského součinu domén (tabulka: schéma + tělo). - *Primární vs. cizí klíč?* □ PK jednoznačně identifikuje řádek (NOT NULL); FK odkazuje na PK jiné tabulky.

SQL □ [[#SQL]] - *Rozdělení příkazů?* □ DDL, DML, DQL, DCL, TCL. - *JOINy?* □ INNER (shoda), LEFT/RIGHT OUTER (zachová jednu stranu), CROSS (kartézský součin). - *Relační algebra* □ *SQL?* □ projekce=SELECT, selekce=WHERE, sjednocení=UNION, průnik=INTERSECT, rozdíl=EXCEPT.

Transakce □ [[#Transakce (ACID)]] - *ACID?* □ Atomicity, Consistency, Isolation, Durability.

Plné znění (ke studiu)

Reprezentace a uložení strukturovaných dat

Nestrukturovaná data (celočíselná, reálná, znaky, řetězce, datum - záleží jak uložené, čas, výčtové typy) mají často význam jen pokud je ukládáme strukturovaně. Existují základní dva způsoby, jak strukturované datové typy vytvářet. Jedná se o **strukturu** a **kolekci**.

Struktura (prostá struktura)

Struktura je tvořena **pevným počtem pojmenovaných dílčích hodnot** (dvojice jméno a hodnota) obecně **různých datových typů**. Jedná se o **uspořádanou n-tici**, jejíž prvky jsou prvky kartézského součinu více množin. Často **strukturu** nazýváme také jako **záznam**. Při ukládání dat v (operační) paměti pomocí struktury se názvy jednotlivých dílčích hodnot neukládají (pracuje se s offsetem), názvy se připojují až při serializaci.

Kolekce

Kolekce je tvořena **předem neomezeným** (tj. proměnným) počtem hodnot **stejného datového typu**. Matematicky se jedná o **množinu**, respektive **multimnožinu** (hodnoty se mohou opakovat), protože obvykle chceme umožnit vícenásobné uložení stejného prvku. **Kolekci** také často nazýváme jako **seznam**, **posloupnost** nebo **řetězec**. Do kolekce můžeme **vkládat** prvky, **získávat** jejich hodnoty a **mazat** je. K tomu používáme **iterátor** (ukazovátka do kolekce). Dále nad kolekcí můžeme provádět souhrnné operace jako získání počtu prvků, průměrné hodnoty, největší, nejmenší, ... případně můžeme iterovat přes všechny prvky. Nad kolekcí může existovat jedno nebo více definovaných uspořádání podle klíčů jejich prvků.

Objekt

Objekt je struktura, kterou lze jednoznačně identifikovat - je mu přiřazena jednoznačná identifikace (**OID** - object identification). Tuto skutečnost využíváme při ukládání do databází, OID většinou **generuje SRBD**. Díky OID může objekt vystupovat ve vztazích.

Ukládání strukturovaných dat

Strukturovaná data jsou obvykle tvořena různě **zanořenými strukturami** a **kolekcemi**. Mohli bychom je do souboru ukládat binárně, tak jak jsou uloženy v operační paměti a informace, co a jak (tj. **metadata** o datech) je v tomto souboru uloženo nějak jinak např. do dalšího souboru. Tento způsob je ale dost nepraktický, proto používáme **standardizované formáty** uložení (**JSON**, **XML**, **YAML**, **TOML**, ...), ve kterých jsou uloženy i (některá) metadata o ukládaných datech (jejich **názvy**, jiná metadata jsou ukládána např. pomocí **Document Type Definition** (DTD) a **XML Schema Definition** (XSD) v případě XML a **JSON-LD** v případě JSON). A formát těchto souborů (**meta²data**) jsou právě popsána standardem (specifikací standardu) a již se nemusí s daty předávat.

Serializace

Serializace (marshalling) je proces **konvertování** datových **struktur** nebo **stavů objektů** do formátu, který je čitelný strojově (někdy i člověkem, ale není to podmínka) a může být **uložen na disk** nebo **přenášen po síti**.

Deserializace

Při deserializaci provádíme **rekonstrukci** serializované hodnoty na **tentýž nebo i jiný formát** (např. serializovaný objekt s určitým pojmenováním atributů můžeme deserializovat do jiného objektu, který má ale stejně pojmenované atributy, ale může mít nějaké další nebo nějaké mohou chybět).

Relační datový model

Relační databáze jsou založené na definicích **množin**, **kartézského součinu** a **relací**, jak je známe z matematiky. **Relace je podmnožinou kartézského součinu** a v případě relačních databází ji chápeme jako **tabulku**. Konkrétně tabulku tvoří **schéma relace** (záhlaví tabulky - názvy sloupců) a **tělo relace** (představuje uložená data v tabulce po řádcích). Počet atributů relace označujeme jako **stupeň** (řád) relace, kardinalita těla relace (počet řádků) označujeme jako **kardinalitu relace**. Postup odvození tabulky z matematických definic: [[media/szz-36/media/image7.png]]

- **Atribut relace** považujeme za sloupec tabulky, [[media/szz-36/media/image9.png]]

[[media/szz-36/media/image1.png]]

- **N-tici relace** považujeme za řádek tabulky.
- **Doména** - Množina hodnot, kterých může atribut nabývat. Hodnoty musí být pouze **skalární**.

Název **relační model** a **relační databáze** je odvozen od faktu, že relace je základním abstraktním pojmem modelu a jedinou strukturou databáze na logické úrovni.

Schéma relační databáze

Schématem relační databáze nazýváme **dvojici (R, I)**, kde:

- $R = \{R_1, R_2, \dots, R_n\}$ je množina schémat relací,
- $I = \{I_1, I_2, \dots, I_m\}$ je množina integritních omezení.

Integritní omezení

Omezení ukládaných dat do databáze plynoucí z reality.

- **specifická:** pro konkrétní aplikaci (pole může nabývat určitých hodnot, může nabývat pouze určitého počtu znaků, nesmí být NULL, ...)
- **obecná:** platí v **každé databázi (primární a cizí klíč).**

Kandidátní klíč Atribut nebo **složený atribut** pro který platí:

- je **jednoznačný**,
- zároveň je ale **minimální** (nelze už dále redukovat).

Každá relace v teorii relačního modelu má alespoň jeden kandidátní klíč.

Primární klíč Primární klíč je jeden z kandidátních klíčů. Primární klíč je základním prostředkem pro adresování n-tice v relačním modelu (řádek tabulky je jednoznačně identifikován primárním klíčem). **Žádné komponenta primárního klíče nesmí být prázdná NULL.**

Cizí klíč Atribut nebo **složený atribut** relace **R2**, pro který platí:

- každá jeho hodnota je buď **zadaná**, nebo je každá **prázdná**,
- V CK je uložena hodnota, která je **shodná** s nějakou hodnotou primárního klíče **jiné relace R1**.

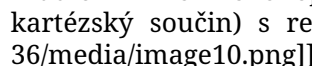
Relační algebra

Relační algebra je dvojice $RA = (R, O)$, kde:

- **R** je množina relací,
- **O** je množina operací (s relacemi)

Množina operací zahrnuje:

- **tradiční množinové operace:** sjednocení, průnik, rozdíl, kartézský součin,
- **speciální relační operace:** **projekce**, **selekce** (restrikce), **spojení** a **dělení**.

Tradiční množinové operace mají stejné výsledky jako u množin, lze ale provádět (až na kartézský součin) s relacemi, které mají stejné schéma (stejně záhlaví tabulky). 

Projekce

Projekce je operace při které **vybíráme jen některé sloupce** z původní tabulky. Z relace **R** tak vytváříme relaci **R[X, Y, ..., Z]** se schématem **(X, Y, ..., Z)** a tělem obsahující patřičné (redukované) **n-tice** odpovídající novému schématu z původní relace **R**. V SQL tomu odpovídá zápis **SELECT name, surname FROM users**, kde users je tabulka obsahující např. (name, surname, title, email, phone, ...). Součástí operace projekce je i **odebrání duplicitních řádků**, které projekcí vzniknou (v SQL je ale nutné použít SELECT DISTINCT, samotný SELECT duplicity neodstraňuje).

Selekce (restrikce)



Restrikce je operace, při které je **zachováno původní schéma** relace (záhlaví tabulky), ale jsou vybrány pouze **některé n-tice** relace (řádky tabulky), které odpovídají určité podmínce. V SQL tomu odpovídá zápis **WHERE id = 42...** (s různými operátory pro porovnání).

Spojení

[[media/szz-36/media/image2.png]]

Spojení je operace, při které je dochází ke **sloučení dvou schémat** relace (dvou záhlaví tabulek) tak, že schéma výsledné relace obsahuje **všechny atributy původních relací** a minimálně jeden atribut je mezi původními relacemi sdílen. Podle tohoto **sdíleného atributu** je prováděno spojení, tak že jsou spojeny n-tice původních relací, které mají stejnou hodnotu tohoto sdíleného atributu. V SQL zapisujeme:

- **INNER JOIN companies ON names.id = companies.id...** Vrací záznamy z levé tabulky které mají **odpovídající** záznam v pravé tabulce (tj. vynechává řádky, které na sebe nelze navázat).
- **LEFT JOIN companies ON names.id = companies.id...** Vrací **všechny** záznamy z **levé tabulky**. K nim připojí odpovídající záznamy z pravé tabulky a pokud žádný odpovídající záznam neexistuje jsou sloupce odpovídající pravé tabulce **prázdné** (ve výsledku nemusí být všechny řádky pravé tabulky).
- **RIGHT JOIN companies ON names.id = companies.id...** Vrací **všechny** záznamy z **pravé tabulky** a připojí k nim odpovídající záznamy z levé tabulky a pokud žádný odpovídající záznam neexistuje jsou sloupce odpovídající levé tabulce **prázdné** (ve výsledku nemusí být všechny řádky levé tabulky).
- **OUTER JOIN companies ON names.id = companies.id...** Vrací všechny záznamy obou tabulek. Pokud mezi nimi neexistuje spojení, jsou atributy patřičné tabulky ve výsledné tabulce prázdné.

Jazyk SQL (Structured Query Language)

[[media/szz-36/media/image11.png]]

SQL je jazyk pro dotazování nad **relačními databázemi**. SQL je standardizovaný, nicméně se některé příkazy mohou lišit napříč implementacemi (jedná se ale o nestandardní části). Jazyk je **case insensitive**.

Definice dat

Tvorba databázových objektů, **tabulky, pohledy, indexy**.

```
CREATE TABLE CREATE TABLE Persons (  
  PersonID int GENERATED AS IDENTITY PRIMARY KEY,  
  LastName varchar(255),  
  FirstName varchar(255),  
  Address varchar(255),  
  City varchar(255)  
);  
  
CREATE TABLE new_table_name AS  
SELECT column1, column2,...  
FROM existing_table_name  
WHERE ....;
```

CREATE INDEX CREATE (UNIQUE) INDEX *index_name*
ON *table_name* (*column1*, *column2*, ...);
CREATE INDEX idx_lastname
ON Persons (LastName);

CREATE VIEW CREATE VIEW *view_name* AS
SELECT *column1*, *column2*, ...
FROM *table_name*
WHERE *condition*;

ALTER ALTER TABLE *table_name*
ADD *column_name datatype*;
ALTER TABLE *table_name*
DROP COLUMN *column_name*;
ALTER TABLE *table_name*
ALTER COLUMN *column_name datatype*;
ALTER TABLE *table_name*
MODIFY COLUMN *column_name datatype*;

DROP DROP TABLE *table_name*;

Manipulace s daty

Při manipulaci s daty jsou operandem báze tabulky nebo pohledy, výsledkem tabulka.

SELECT

INSERT [[media/szz-36/media/image5.png]]
[[media/szz-36/media/image12.png]]
[[media/szz-36/media/image6.png]]
[[media/szz-36/media/image8.png]]
[[media/szz-36/media/image3.png]]
INSERT INTO *table_name* (*column1*, *column2*, *column3*, ...)
VALUES (*value1*, *value2*, *value3*, ...);
INSERT INTO *table_name*
VALUES (*value1*, *value2*, *value3*, ...);
INSERT INTO Customers (CustomerName, ContactName, Address, City, PostalCode, Country)
VALUES ('Cardinal', 'Tom B. Erichsen', 'Skagen 21', 'Stavanger', '4006', 'Norway');

```
UPDATE  UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;

UPDATE Customers
SET ContactName = 'Alfred Schmidt', City= 'Frankfurt'
WHERE CustomerID = 1;
```

```
DELETE  DELETE FROM table_name
WHERE condition;

DELETE FROM Customers
WHERE CustomerName='Alfreds Futterkiste';
```

Pohled (View)

Virtuální DB struktura (v pohledech nejsou uložena data), může zprostředkovávat data z nula a více tabulek. Pracuje se s nimi jako s tabulkami. Mohou z tabulek vybírat jen určité řádky nebo sloupce, mohou obsahovat výrazy, mohou spojovat data z více tabulek, mohou odkazovat na další pohledy. Používá se pro zjednodušení práce (opakované dotazování). Vytváří se pomocí dotazu SELECT.

Kurzor (Cursor)

Prostředek, který umožňuje **sekvenčně zpracovávat data** - číst i modifikovat. Umožňuje např. upravovat jednotlivé řádky tabulky imperativním způsobem na místo deklarativního způsobu jako u dotazu UPDATE.

Uložená procedura (Stored procedure)

Uložená procedura (rutina) je **sada příkazů SQL**, které jsou uloženy na databázovém serveru a vykonává se tak, že je zavolána prostřednictvím dotazu názvem, který jim byl přiřazen (je to určitá obdoba funkce).

Trigger

Trigger je uložená procedura, který se spustí automaticky jako reakce na určitou akci s danou tabulkou (před nebo po ní). Triggery nastavujeme na dotazy UPDATE, INSERT, DELETE.

DB transakce

Databázová transakce poskytuje mechanismus, který zajišťuje, že při manipulaci s daty v databázi bude databáze setrvávat v **konzistentním stavu**. Databázová transakce splňuje vlastnosti, které jsou známy pod akronymem **ACID**. Jednopříkazové operace nemusí být obaleny transakcí, databázové systémy k nim přistupují jako k transakcím.

Atomicita (Atomicity) Databázová transakce je již dále nedělitelná. Tzn. buď se provedou všechny její části (příkazy), nebo se neprovede žádný. Např. při vkládání dat do DB, která mají být uložena do více tabulek se musí vždy uložit všechny, nebo operace selže tak, že se neuloží žádná.

Konzistence (Consistency) Transakce musí **zachovat konzistenci** databáze, tzn. **nesmí být porušeno** žádné ze specifických ani obecných **integritních omezení**.

Izolovanost (Isolation) Operace prováděné v rámci nedokončené transakce **nemohou ovlivnit jiné probíhající** transakce (např. pokud dojde k rollback - jiná transakce nemůže použít data, která byla upravena touto transakcí, protože úpravy mohou být anulovány. Případně lze dovolit použití těchto dat, ale anulovat všechny transakce, které je používaly). Musí být zajištěno, že **transakce probíhají jedna po druhé, pokud manipulují se stejnými daty**.

Trvalost (Durability) Poté co je transakce dokončena (commit), je zajištěno, že provedené změny budou mít **trvalý charakter**, a to **i při výpadku systému**.

Business transakce

Jako business transakce chápeme běžné operace, které provádí uživatelé v informačním systému. Např. **vytvoření faktury, nákup zboží v eshopu, vystavení dokladu, příjem zboží** atd. Vytvoření faktury může znamenat zadání nákupčího, data splatnosti, položek faktury a její odeslání emailem nebo vytisknutí. Tato business transakce je tvořena několika databázovými transakcemi, které v průběhu realizace business transakce zajišťují konzistenci v databázi. Např. **přidání každé položky faktury může představovat novou DB transakci** (stav faktury s 1 nebo 2 položkami je konzistentní, nekonzistentní by bylo, kdyby se nějaká položka uložila pouze částečně). Systémy, které se zabývají transakčním zpracováním se označují jako **On-Line Transaction Processing (OLTP)** systémy.

Zdroje

- SZZ okruh 36 — studijní materiály FIT BUT (szz-36.docx). Obrázky: media/szz-36/.

Navigace: [[index| • Přehled okruhů]] · □ [[okruhy/35-konceptualni-modelovani-db|35. Konceptuální modelování a návrh DB]] · další: [[okruhy/37-webova-rozhrazi-autentizace|37. Webová rozhraní a autentizace]] □